



PowerShell: Empower the Windows CLI

Kshitij Sobti

Serious command line interface (CLI) advocates always shy away from Windows, deterred by the weak command interface it provides, and the lack of a pure CLI mode. With Windows PowerShell, Microsoft has brought power to the console, and given some geeks a road back to Windows.

Until Windows 98, we were used to an option to go into the pure console mode if all things fell apart. The presence of DOS in the event everything else failed was always comforting. Microsoft seems to have abandoned its CLI since then, with only a few minor enhancements, 'command.com' became 'cmd.exe' but significant changes were yet to come. Compared to the advanced nature of commands available for *nix environments, Windows can be said to effectively have no CLI.

Microsoft tried to give developer features such as Windows Script Host (WSH) to enable automating tasks, and scripting redundant procedures. However, it only managed to open numerous security backdoors, and become more associated with viruses than anything else. Finding the WSH process in your process list was a trigger of panic.

Microsoft fortunately hasn't given up. Windows PowerShell is its cure for the Windows CLI. Released in November 2006, Windows PowerShell is truly a powerful CLI option. Although it is still in its infancy with just one release, it has managed to live up to its name.

The CLI really?

Many people swear by the command line interface and sing praises of how much faster it performs some complex tasks. However, for most users, graphical user interfaces have become indispensable. Command lines may make complex tasks achievable. However, the truth often is that GUIs make simpler tasks easier. So why bother with a CLI?

Often we need to perform simple tasks that have rather trivial command line solutions. For example, how would you rename all the '.abc' files in a folder to '.xyz'? While the command line solution is a simple 'ren *.abc *.xyz' executed in the said directory, the GUI by default offers no such comfort. How about renaming all the files in a directory which are greater than a fixed size? While such a thing would have required a complicated script before, with PowerShell, all this can be done in one line! Don't believe us! Read on and find out.

Setting up PowerShell

PowerShell is a free download available from the Microsoft web site, and is available for Windows XP onwards. The current stable version is V1. However, Microsoft has also released a CTP3 (Community Technology Preview) of its upcoming V2. Both versions are included on this month's DVD for your convenience.

Once you start the console, you will realise that it is quite similar to your normal command line, except that the colour scheme is different, with a blue backdrop and white text, and with a 'PS' in

the beginning of the prompt. This is where all the magic happens.

Cmdlets

Windows PowerShell introduces the concept of cmdlets, which are basically small commands that can be run from the command line. They are unique to PowerShell, in that they won't run outside a PowerShell Window. Similar to normal Windows CLI commands, you run them by typing the command name, along with parameters to control their functioning.

In PowerShell, all commands follow the same naming convention of Verb-Noun, making the command names longer, but still easy to memorise. For example, some common commands are Get-Content, Move-Item, Rename-Item and Format-List.

In PowerShell, commands do not directly output the result to the screen, but instead return all the information as a .NET object, which PowerShell then displays. This allows for further processing of the output before it is finally written to the screen. PowerShell comes with many output formatting cmdlets for this purpose.

A very important and powerful new concept in PowerShell is pipelining, a concept with which DOS users will be familiar. Pipelining allows you to pass the output of one command as the input to another. Something like the example given above could be done with three commands:

- A command to get the directory listing
- A command to filter out the files to be renamed
- A command to rename the files

We pass the directory listing to the filter, and the filtered results to the renaming command, forming a 'pipeline' of sorts.

So what would this command look like in PowerShell?

```
PS> Get-ChildItem | Where-Object {$_.length -gt 50kb} | Rename-Item -newname {$_.name + '.big'}
```

This will rename all the files in the current folder which have a size greater than 50 kB, and add an extension of '.big'.

This might seem an unnecessary way of running commands. However, PowerShell provides multiple aliases for each command, so that the same result as above could have also been achieved by:

```
PS> ls | where {$_.length -gt 50kb} | ren -new {$_.name + '.big'}
```

Almost all commands have short forms for faster typing. However, while writing a script that will be used later, using the full command name can be helpful for future readers.

The command we just saw is divided into three different parts, separated by the vertical bar (aka. the pipe character).

```
Get-ChildItem {$_.length -gt 50kb}
```

This is similar to the 'dir' command. However, it is also capable of listing registry entries. You can also have a parameter such as '*.xyz', to display all '.xyz' files, or use the '-recurse' parameter to get a recursive list of all the files in the folder and its child folders. This output is being piped on to:

- Where-Object

Now this immediately highlights the difference between the regular commands you are used to, and PowerShell. The 'Where-Object' command filters the input it gets from 'Get-ChildItem' based on the conditions provided in the parameters.

Here the condition enclosed in braces, '{\$_.length -gt 50kb}', specifies that for each object the required condition is that its size should be greater than 50 kB. Here, '\$_' represents the object being tested for the condition. By using '\$.Length', we obtain the size or length of the Object, and since the object we are considering here is a file, the length is basically the file size. The final piece is '-gt 50kb', which means greater than 50 kB.

